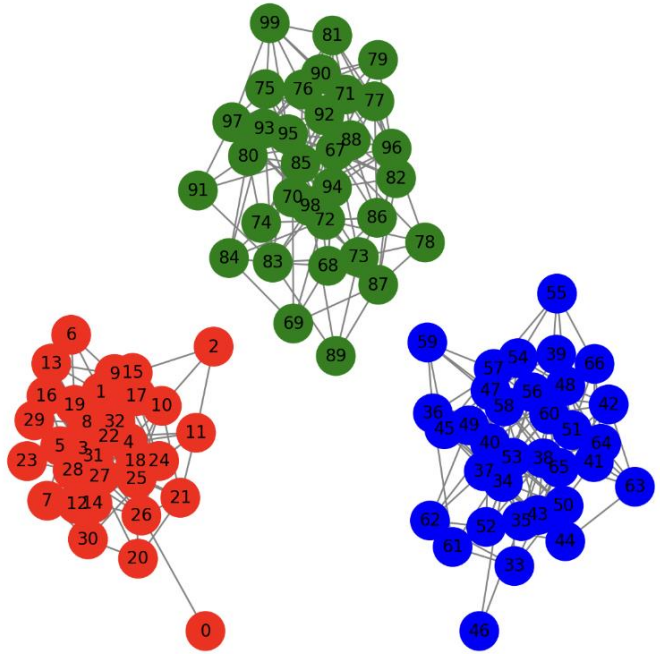
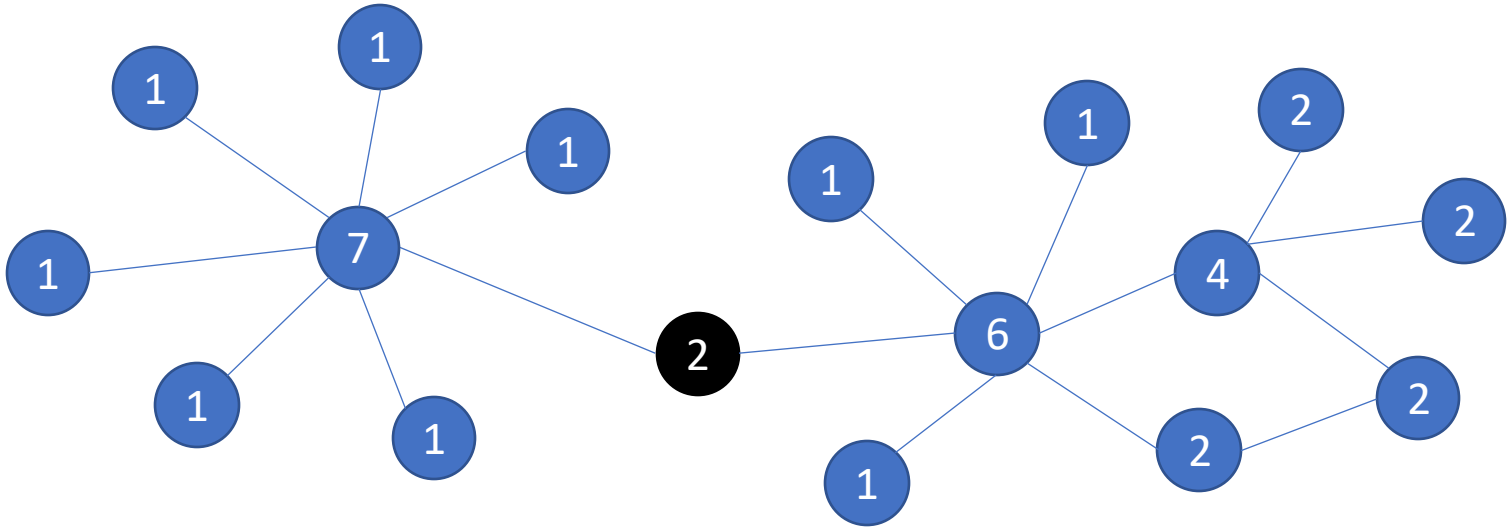
Review: Social Network Analysis and Web Scrape

Review

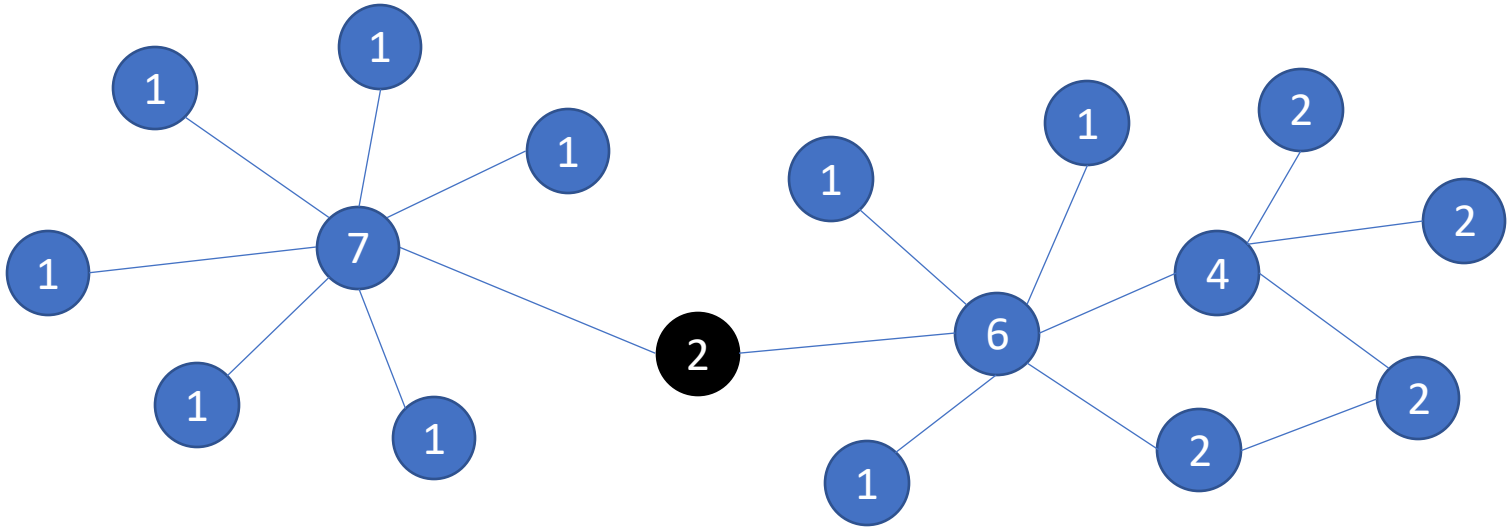
- Detect unconnected communities
- Detect connected communities
  - by removing the "broker" type of edges





Stoer-Wagner algorithm

- Connectivity: the minimal number of edges/nodes to dissect a network
- Identify and remove such nodes/edges
- Stoer-Wagner algorithm finds the global minimum cut, meaning the smallest set of edges that disconnects any part of the network.
- Limitation: most likely to be only 2 communities that are not necessarily well connected



Python

- ```
  ##Stoer-Wagner algorithm
  cut_value, partition = nx.stoer_wagner(G)

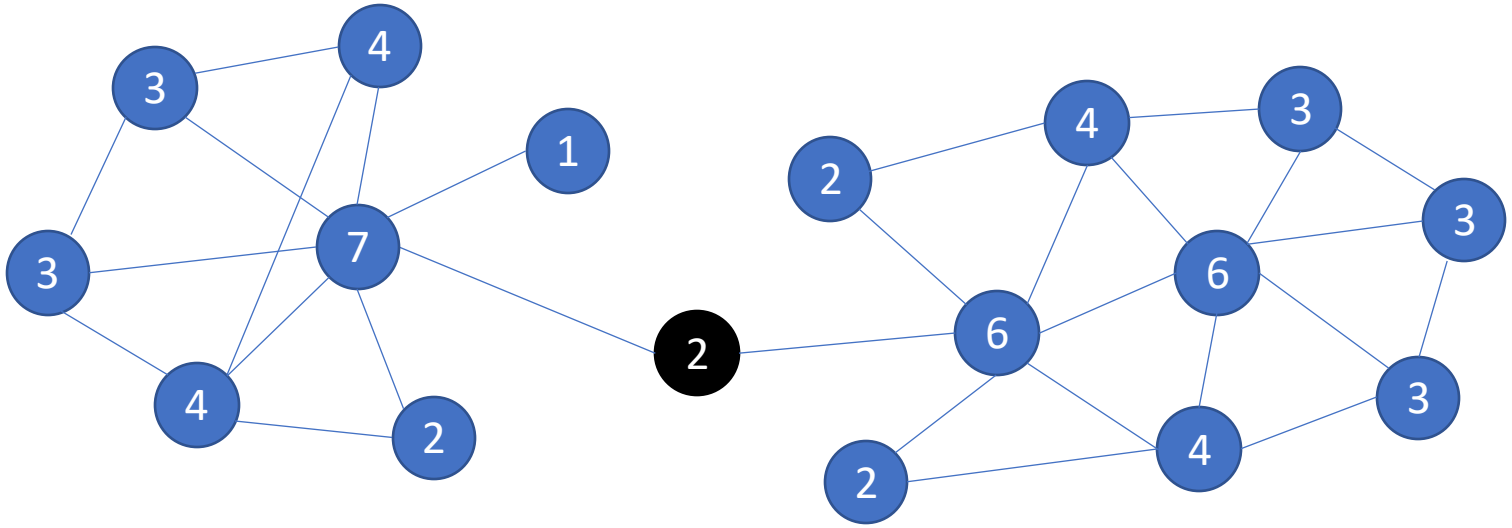
  # Print results
  print("Minimum Cut Size:", cut_value)
  for i, component in enumerate(partition):
      print(f"Component {i + 1}: {sorted(component)}")
  ```
  ```

k-Core decomposition

- Goal: identify densely connected communities
- Approach: wash off sparsely connected nodes, i.e., low-degree nodes
- The broker tends to be node with low degree
- Result: A **k-core** is a maximal subgraph where each node has at least k connections

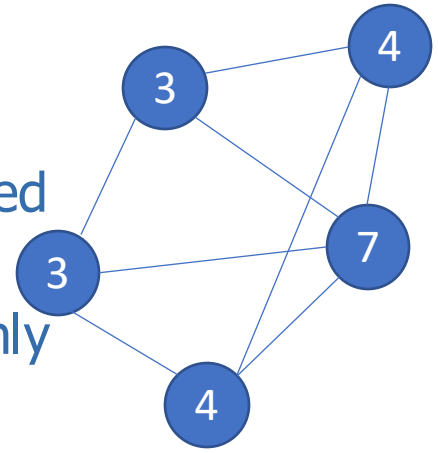
k-Core decomposition

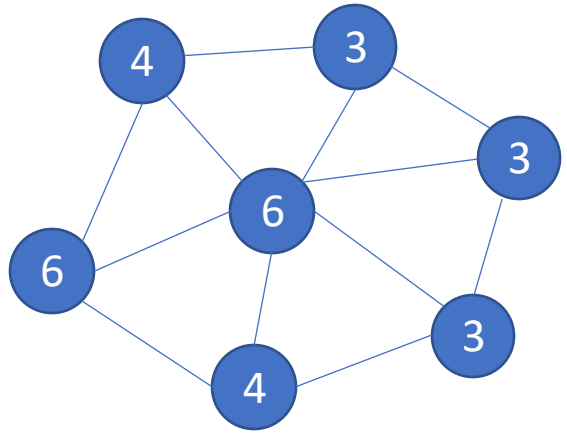
- Choose a k value (starting from k=1).
- Remove all nodes with degree < k.
- Repeat until the remaining nodes all have degree ≥ k.
- Increase k and repeat until the network is fully dissected.

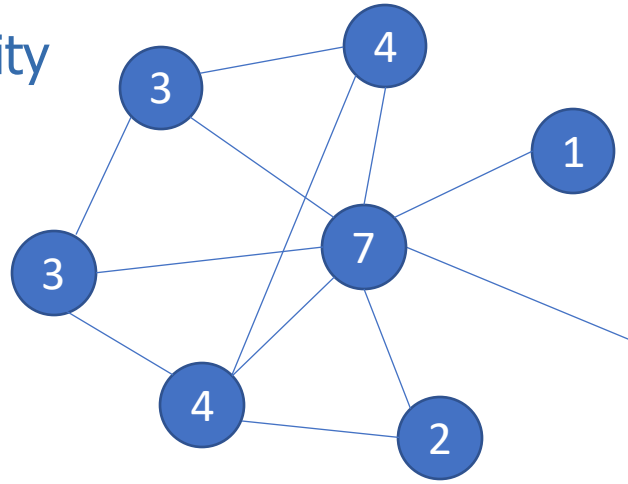


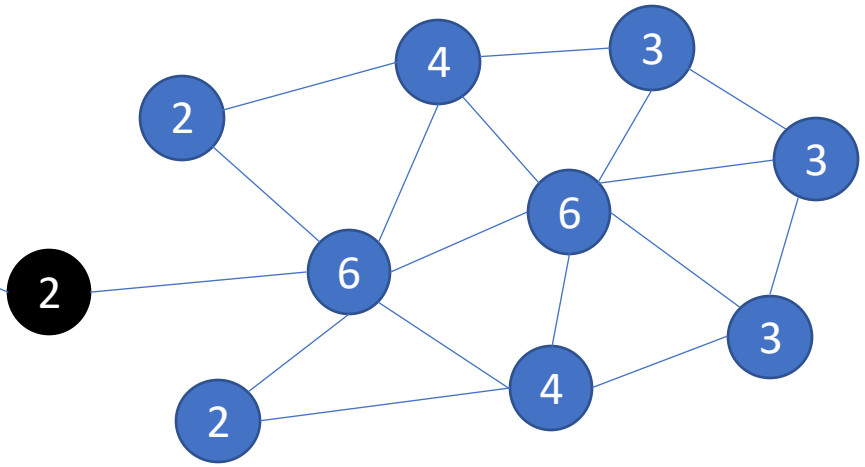
Functionality

- Not for dissecting a network
- Identifies the most densely connected and robust parts of the network
- The "backbone" of the network – only the strongest connections remain
- You can choose k to your discretion
- Limitation: may wash off the majority of the networks and identify 1 or even 0 community









Python

```
k_core = nx.k_core(G, k=2)  # k=2 means every node must have at least 2 connections
components_in_k_core = list(nx.connected_components(k_core))

# Print k-core nodes
print("Nodes in the 2-core:", list(k_core.nodes()))

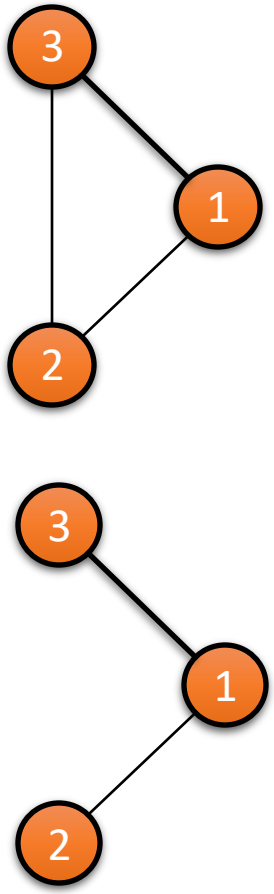
# Display the detected components
for i, component in enumerate(components_in_k_core):
    print(f"Component {i+1}: {sorted(component)}")
```

Louvain Algorithm

- A hierarchical method for community detection in networks.
- Based on modularity optimization
- Modularity: measures the density of connections inside communities compared to connections between them.

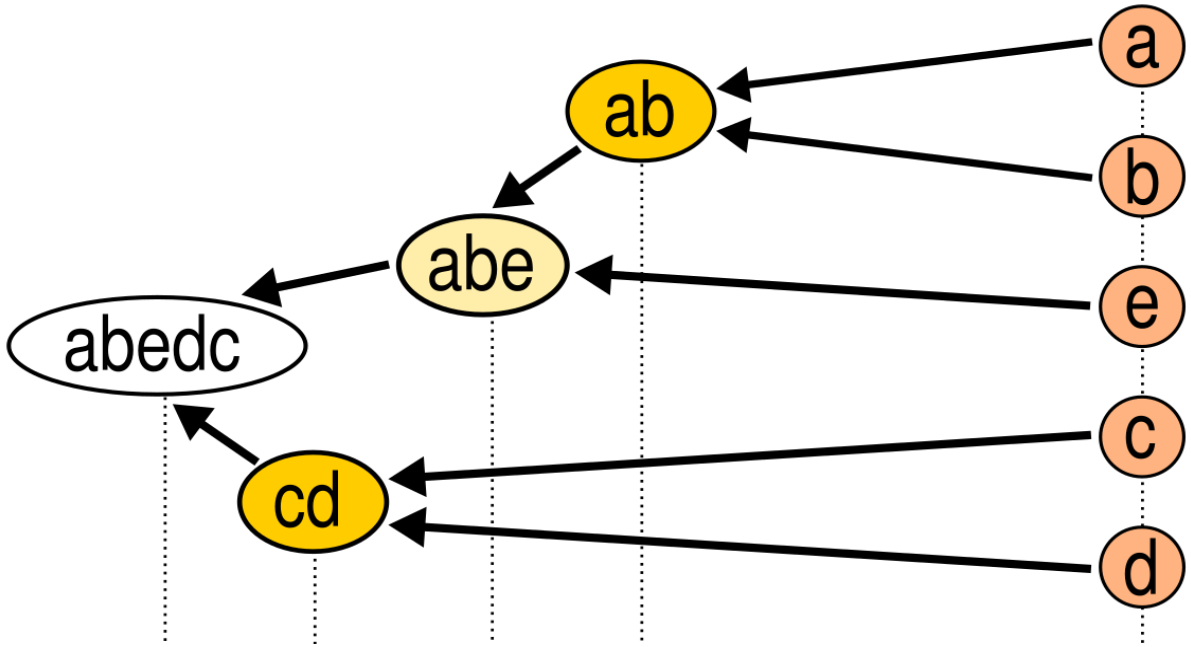
$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j)$$

- **A_ij**: Adjacency matrix (1 if edge exists, 0 otherwise).
- **k_i, k_j**: Degrees of nodes **i** and **j**.
- **m**: Total number of edges in the graph.
- **δ(c_i, c_j)**: 1 if nodes **i** and **j** belong to the same community, otherwise 0.



Process

- **Initialization**: Assign each node to its own community.
- **Phase 1 - Modularity Optimization**:
  - Move a node to a neighboring community to maximize modularity.
- **Phase 2 - Community Aggregation**:
  - Collapse detected communities into super-nodes.
- **Repeat** phase 1 and 2 iteratively until no improvement is possible.



Python

```
# Install using 'pip install python-louvain
import community.community_louvain as community_louvain

# Apply Louvain algorithm for community detection
partition = community_louvain.best_partition(G)

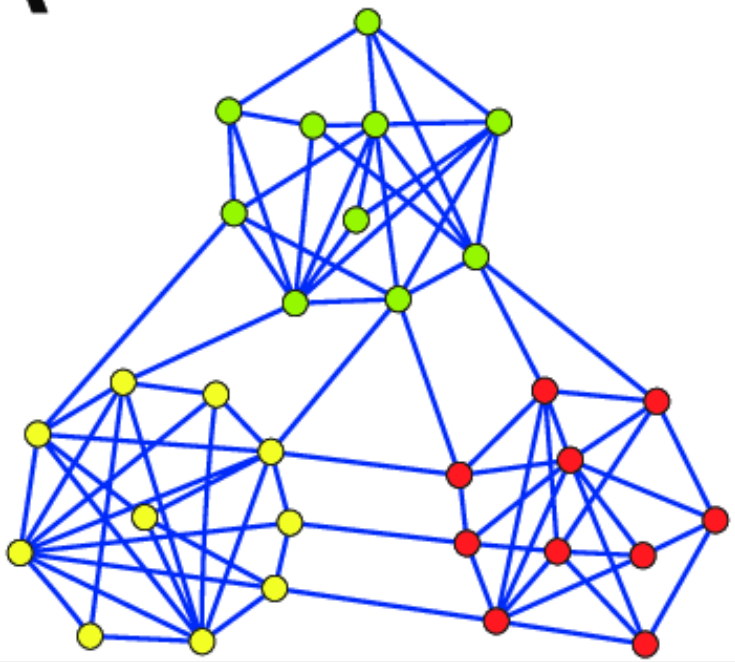
# Print detected communities
communities = {}
for node, comm in partition.items():
    if comm not in communities:
        communities[comm] = []
    communities[comm].append(node)

for comm, nodes in communities.items():
    print(f"Community {comm+1}: {sorted(nodes)}")
```

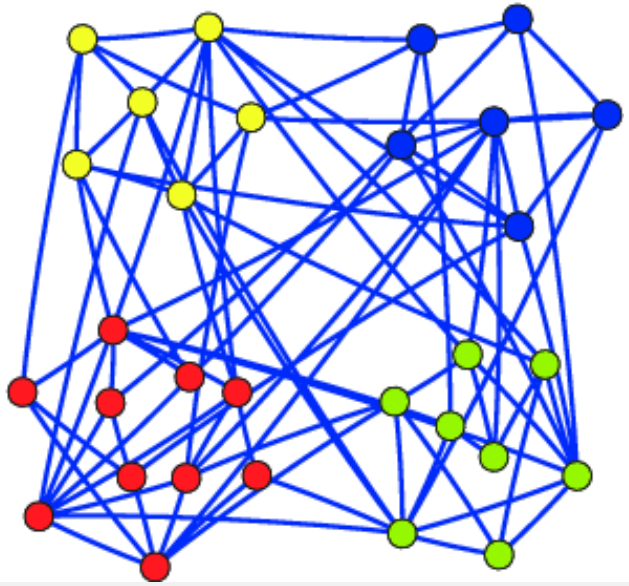
Indicator of network communities

- While you can always apply network partitioning
- It doesn't mean the partitioning results make good sense
- This can be reflected by the final modularity score

A Q=0.7



B Q=0.2



Modularity score

- Post-partitioning modularity scores

Modularity (Q)	Community Structure Strength
$Q < 0.1$	No significant communities (almost random graph).
$0.1 \leq Q < 0.3$	Weak community structure.
$0.3 \leq Q < 0.5$	Moderate to strong community structure.
$Q \geq 0.5$	Well-defined, strong communities.
$Q \approx 1$	Highly modular, almost disconnected groups.

Pre-partitioning indicators

- Cohesion

- Centralization

Centralization Type	Measures	Key Question
Degree Centralization	Node Degree	Is the network controlled by a few well-connected nodes?
Betweenness Centralization	Shortest Paths	Do a few nodes act as key brokers?
Closeness Centralization	Distance to Others	Are some nodes much closer to all others?
Eigenvector Centralization	Influence	Do a few nodes dominate in influence?

MGP 6b

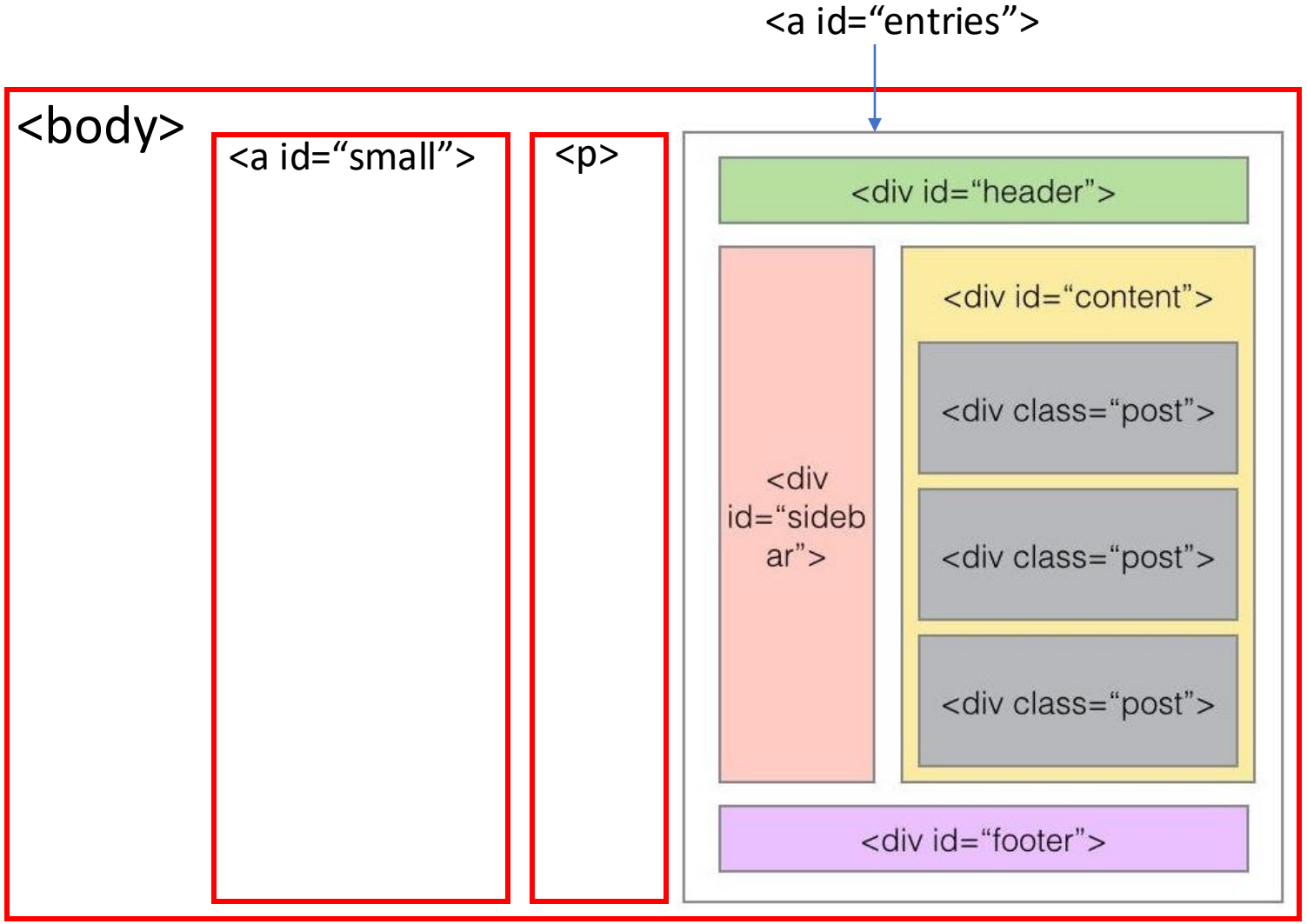
- For your good performance, your client want to collaborate with you on a new campaign. This campaign launches some community-level promotion. In specific, they want to find some communities and then design promotional strategies that are tailored to them. (e.g., differentiated strategies for Chinese, Turkish, Italian, or Vietnamese communities, knowing that they all have fair amount of presence in the scope of networks)
- Among the two networks, which one may better fit this strategy?
- Please help the campaign manager to find such communities using Girvan-Newman, Stoer-Wagner, and Louvain methods

Review

- Mod 1: Web Scrape
- 12 questions

- Mod 2: Social Network Analysis
- 18 questions

Web scrape: HTML Structure



- <html>
  - <head>
  - <body>
    - <a id="small">
    - <p>
    - <a id="entries">
      -
      -
      -
      -
        -
        -
        -
      -

- Class and id

Scrape one (repeated) feature on one page

- Selector: response.xpath()
  - xpath specifications (full and short cut)
  - extract vs. extract_first
- Commander line operations

<html>
  <head>
  <body>
  - <a id="small">
  - <p>
  - <a id="entries">
    -
    -
    -
    -
      -
      -
      -
    -

```
def parse(self, response):
   quotes = response.xpath('//body/a[@id="entries"]/div[@id="sidebar"]div[@class="post"]/text()').extract()
```

```
def parse(self, response):
   quotes = response.xpath('//div[@class="post"]/text()').extract()
```

Scrape more than one (repeated) features

+ Building wrapper
  - Advantage of using wrapper?
  - Selector specification under wrapper
  - For loop

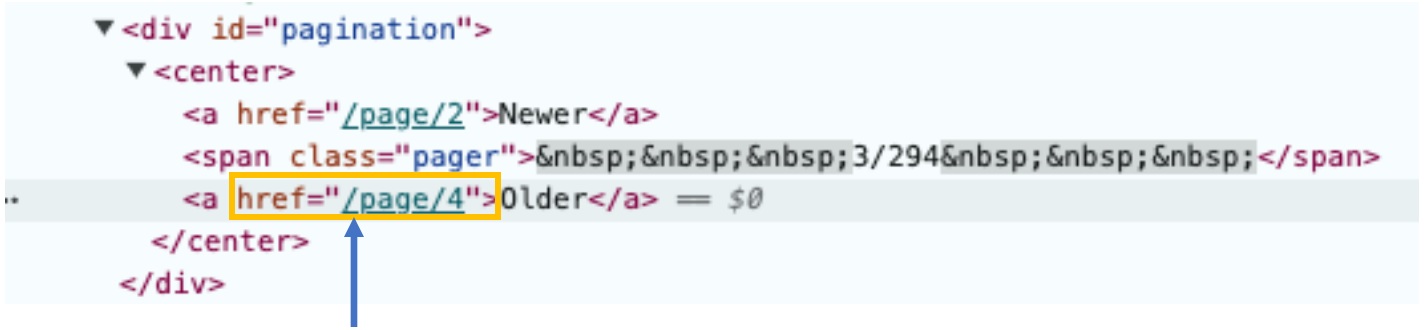
-
  - QUOTE
  - AUTHOR
  -
  - …
-
  - QUOTE
  - AUTHOR 1
  - AUTHOR 2
  -
  - …

Quote 1	Author 1
Quote 2	Author 2x
Quote 3	Author 2y
Quote 4	Author 3

```
def parse(self, response):
    wrappers = response.xpath('//div[@class="post quote"]')
    for wrapper in wrappers:
        quote = wrapper.xpath('span/text()').extract_first()
        author = wrapper.xpath('div[@class="source"]/text()').extract_first()
        yield {'Quote': quote, 'Author': author}
```

Scrape with pagination (across pages of the same type)

+ Browse: Request()
  - Obtain hyperlink (relative vs. absolute)
  - For loop: callback

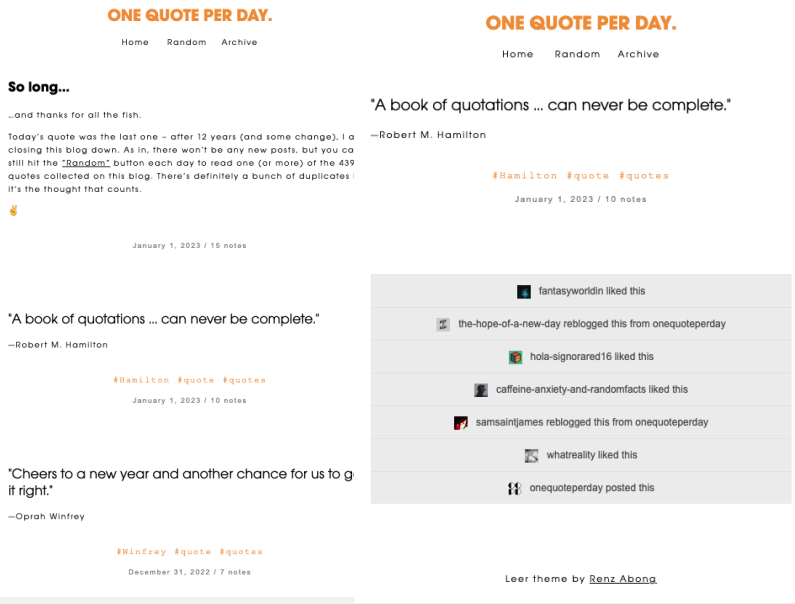



```
def parse(self, response):
    wrappers = response.xpath('//div[@class="post quote"]')
    for wrapper in wrappers:
        quote = wrapper.xpath('span/text()').extract_first()
        author = wrapper.xpath('div[@class="source"]/text()').extract_first()
        yield {'Quote': quote, 'Author': author}
```

```
    next_rel_url = response.xpath ('//div[@id="pagination"]/center/a/@href').extract()[-1]
    next_url = response.urljoin(next_rel_url)
    yield Request(next_url, callback=self.parse)
```

Scrape hierarchical pages (of different types)

+ Multiple parse functions (one for each type)
  - Request()
  - callback
  - meta




```
def parse(self, response):
        wrappers = response.xpath('//div[@class="post quote"]')
        for wrapper in wrappers:
                quote = wrapper.xpath('span/text()').extract_first()
                author = wrapper.xpath('div[@class="source"]/text()').extract_first()
                lower_url = wrapper.xpath('div[@class="post_footer"]/a/@href').extract_first()
                yield Request(lower_url, callback= parse_lower, meta={'Quote': quote, 'Author': auth
        next_rel_url = response.xpath ('//div[@id="pagination"]/center/a/@href').extract()[-1]
        next_url = response.urljoin(next_rel_url)
        yield Request(next_url, callback=self.parse)
```

```
def parse_lower(self, response):
        notes = response.xpath('//span[@class="action"]/a/text()').extract()
        response.meta['Notes'] = notes
        yield response.meta
```

Question types

- Which of the following XPath expressions correctly selects all <a> elements that are direct children of a element with the class "content"?
- A) //div[@class="content"]/a
- B) //div[@class="content"]//a
- C) /div[@class="content"]/a
- D) //a[@class="content"]/div

- What is the correct way to send a request to "https://example.com/data" and process the response using the parse_data callback function?
- A) yield request(url="https://example.com/data", callback=parse_data)
- B) yield Request(url="https://example.com/data", callback=self.parse_data)
- C) yield Get(url="https://example.com/data", callback=parse_data)
- D) return Request(url="https://example.com/data", callback=parse_data)

Social network: Basic concepts

- Network vs. social network vs. social media network
- Nodes
- Edges
- Directedness
- Weights
- Node attributes
- Edge attributes
- Walk, path, cycle, geodesic
- Geodesic distance
- Components
- Strongly connected networks
- Neighborhood

Representation

- Representation
  - Adjacency list
  - Adjacency matrix
  - Edge list
  - Code for attributes
- Compute basic metrics based on adj matrix?
  - No. nodes
  - No. undirected edges
  - No. directed edges
- Understand the codes

Node	Head to
1	2
2	
3	2,4
4	1

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

(1,2)
(4,1)
(3,4)
(3,2)

```
adj_list = {
    "Alice": ["Bob", "Charlie"],
    "Bob": ["Alice", "David"],
    "Charlie": ["Alice", "Eve"],
    "David": ["Bob", "Eve"],
    "Eve": ["Charlie", "David"]
}
G = nx.Graph(adj_list)
```

```
adj_matrix = np.array([
    [0, 1, 1, 0, 0],  # Alice
    [1, 0, 0, 1, 0],  # Bob
    [1, 0, 0, 0, 1],  # Charlie
    [0, 1, 0, 0, 1],  # David
    [0, 0, 1, 1, 0]   # Eve
])
G = nx.from_numpy_array(adj_matrix)
```

```
edges = [
    ("Alice", "Bob"),
    ("Alice", "Charlie"),
    ("Bob", "David"),
    ("Charlie", "Eve"),
    ("David", "Eve")
]
G = nx.Graph(edges)
```

Network-level metrics: Cohesion

Cohesion Metric	Definition	Calculation	Interpretation & Implications	Python Code	Uses
Density	The ratio of actual edges to possible edges in a network.	Density = 2 * |E| / (|V| * (|V| - 1))	Higher density → more connections, less fragmentation.	nx.density(G)	Measures overall network cohesiveness.
Average Degree	The average number of connections per node in the network.	Avg Degree = sum(degree of all nodes) / |V|	Higher average degree → more robust and interactive network.	sum(dict(G.degree()).values()) / len(G.nodes())	Indicates the interaction level of a network.
Clustering Coefficient	The likelihood that a node's neighbors are connected to each other.	Global Clustering Coefficient = (number of closed triplets) / (number of triplets)	Higher clustering → stronger local cohesion and community structure.	nx.transitivity(G)	Shows tight-knit groups in networks; applied in community detection and recommendation systems.
Diameter	The longest shortest path between any two nodes in the network.	Diameter = max(shortest path length between all node pairs)	Higher diameter → more spread-out network; lower means more efficiency.	nx.diameter(G)	Determines the efficiency of communication and travel routes.
Connectivity	The minimum number of nodes or edges that need to be removed to disconnect the network.	Node Connectivity = min number of nodes to remove to disconnect the graph; Edge Connectivity = min number of edges to remove.	Higher edge connectivity → more robust to edge failures.	nx.node_connectivity(G) nx.edge_connectivity(G)	Evaluates network robustness and vulnerability.
Reciprocity	Measures the proportion of mutual connections in a directed network.	Reciprocity = number of reciprocal edges / total directed edges.	Higher reciprocity → more mutual interactions	nx.reciprocity(G) # Only for directed graphs	Analyzes mutual connections in social, communication, and trade networks

Network-level metrics: Centralization

Centralization Measure	Definition	Calculation	Interpretation & Implications	Python Code	Uses
Degree Centralization	Measures the variation in degree centrality across the network.	Sum of differences between the most connected node's degree and all other nodes, normalized.	Higher values indicate a more star-like structure; one node dominates connectivity.	nx.degree_centrality(G), then compute network-wide extremity.	Identifies networks dominated by a few highly connected nodes.
Betweenness Centralization	Measures the variation in betweenness centrality across the network.	Sum of differences between the highest betweenness node and all other nodes, normalized.	Higher values indicate that information flow is controlled by a few key nodes.	nx.betweenness_centrality(G), then compute network-wide extremity.	Detects networks where information control is concentrated.
Closeness Centralization	Measures the variation in closeness centrality across the network.	Sum of differences between the highest closeness node and all other nodes, normalized.	Higher values suggest that some nodes have significantly faster access to all others.	nx.closeness_centrality(G), then compute network-wide extremity.	Measures accessibility within a network.
Eigenvector Centralization	Measures the variation in eigenvector centrality across the network.	Sum of differences between the highest eigenvector node and all other nodes, normalized.	Higher values mean influence is concentrated in a few well-connected nodes.	nx.eigenvector_centrality(G), then compute network-wide extremity.	Finds influential leaders in hierarchical or power-law networks.

- When and why to conduct network diagnosis?

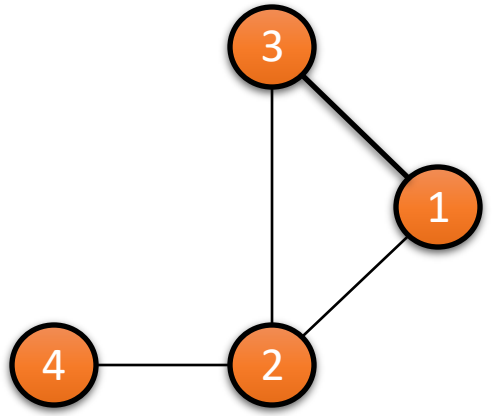
Node-level: Centrality

Centrality Measure	Definition	Calculation	Interpretation & Implications	Python Code	Uses
Degree	Number of direct connections a node has.	Degree of node i: d(i) = number of edges connected to i.	Higher degree → more directly connected, more local influence.	nx.degree_centrality(G)	Identifies highly connected nodes; used in influencer detection and spreading information.
Clustering	Measures how well a node's neighbors are connected.	C(i) = actual links among neighbors / possible links.	Higher clustering → tightly connected local group.	nx.clustering(G)	Measures local group cohesion; used in clustering analysis and identifying tightly knit groups.
Betweenness	Counts how often a node is on the shortest path between other nodes.	Betweenness B(i) = sum of shortest paths passing through i.	Higher betweenness → more control over information flow.	nx.betweenness_centrality(G)	Finds key intermediaries; used in network robustness, information flow, and communication control.
Closeness	Measures how close a node is to all other nodes in the network.	Closeness C(i) = 1 / sum of shortest path distances to all nodes.	Higher closeness → more central, faster access to other nodes.	nx.closeness_centrality(G)	Detects central nodes with quick access to others; used in transportation, navigation, and efficiency analysis.
Eigenvector	Measures influence based on connections to high-degree nodes.	Eigenvector centrality is computed from the adjacency matrix eigenvalues.	Higher eigenvector → connected to important nodes, more influence.	nx.eigenvector_centrality(G)	Identifies the most influential nodes based on network structure; used in ranking, leadership detection, and marketing.

Question types

- Which metric **cannot** help in identifying the most influential nodes in the network?
  A. nx.degree_centrality(G)
  B. nx.betweenness_centrality(G)
  C. nx.transitivity(G)
  D. nx.clustering(G)

- Consider the following simple **undirected network**:



- What is the clustering coefficient of **node 2** in this network?
  A. 0.33
  B. 0
  C. 0.5
  D. 1

Community detection

Method	Definition	Calculation	Interpretation & Implications	Python Code
Girvan-Newman	Detects communities by iteratively removing edges with the highest betweenness centrality until the network splits.	Computes edge betweenness, removes the highest betweenness edge, and repeats until communities emerge.	Detects hierarchical community structures; useful for networks with clear bridges.	from networkx.algorithms.community import girvan_newman comp = girvan_newman(G) communities = next(comp)
Stoer-Wagner	Finds the global minimum cut in the network to partition it into two separate groups.	Uses max-flow min-cut theorem to determine the minimum number of edges that separate the network.	Identifies weak points in networks and partitions them into two dense clusters.	cut_value, partition = nx.stoer_wagner(G)
k-Core	Identifies the largest subgraph where each node has at least k connections, forming a hierarchical structure.	Removes nodes with degree < k iteratively until a stable k-core structure is obtained.	Finds the most interconnected cores, highlighting strong, dense communities.	k_core = nx.k_core(G, k=2)
Louvain	Partitions the network by optimizing **modularity**, iteratively refining community assignments.	Maximizes modularity using greedy optimization by grouping nodes into communities.	Efficient for large-scale networks and finds well-separated, meaningful clusters.	import community.community_louvain as cl partition = cl.best_partition(G)

- Detect components
- Modularity and network-level indicators

Question types

- Which community detection method would result in a loss of some nodes?
- A) **Girvan-Newman Algorithm**
- B) **Stoer-Wagner**
- C) **k-Core Decomposition**
- D) **Louvain Method**.

- Which approach below can also calculate the edge connectivity of a network?
- A) **Girvan-Newman Algorithm**
- B) **Stoer-Wagner**
- C) **k-Core Decomposition**
- D) **Louvain Method**.

That’s all, folks!